

Strategy

Strategie



Hlavní idea

Strategy (Strategie) je behaviorální návrhový vzor (vzor související s chováním), který umožňuje definovat skupinu algoritmů, každou z nich umístit do samostatné třídy a učinit jejich objekty zaměnitelnými.



Návrhový vzor **Strategy** umožňuje za běhu aplikace změnu algoritmu za jiný, a to bez nutnosti změny kódu programu. Algoritmy jsou koncipovány jako jednotlivé moduly. Jde o další návrhový vzor chování skupiny GoF.



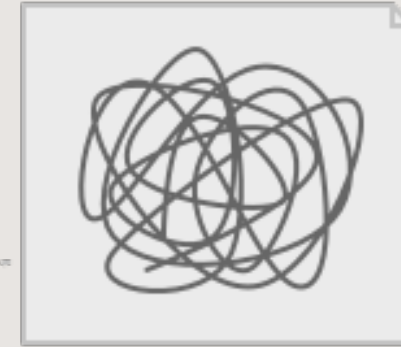
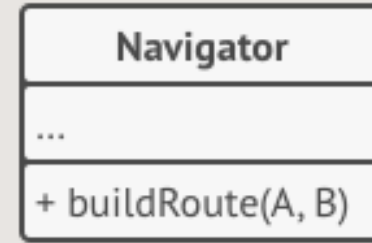
Problém

Jednoho dne jste se rozhodli vytvořit navigační aplikaci pro příležitostné cestovatele. Aplikace byla soustředěna kolem mapy, která uživatelům pomáhala rychle se orientovat v jakémkoli městě.

Jednou z nejžádanějších funkcí aplikace bylo automatické plánování trasy. Uživatel by měl být schopen zadat adresu a na mapě vidět nejrychlejší trasu k tomuto cíli.



Problém



První verze aplikace uměla vyhledat pouze trasy po silnicích. Jak se ale zdá, ne každý rád jezdí na dovolenou autem. Takže s další aktualizací jste přidali možnost vytvářet pěší trasy. Hned poté jste přidali další možnost, jak umožnit lidem používat na svých trasách veřejnou dopravu.

To byl však jen začátek. Později jste plánovali přidat vyhledání trasy pro cyklisty. A ještě později další možnost vyhledávání tras přes všechny turistické atrakce města.



Problém

Zatímco z obchodního hlediska byla aplikace úspěšná, technická část vám způsobila mnoho problémů. Pokaždé, když jste přidali nový vyhledávací/plánovací algoritmus, hlavní třída navigátoru se dvakrát zvětšila. V určitém okamžiku byla náš algoritmus nepřehledný a příliš obtížný na údržbu.

Jakákoli změna jednoho z algoritmů, ať už to byla jednoduchá oprava chyby nebo mírná úprava ohodnocení ulice, ovlivnila celou třídu a zvýšila pravděpodobnost na vytvoření chyby v již fungujícím kódu.



Problém

Týmová práce a kooperace se navíc stala neefektivní. Vaši kolegové, kteří byli najati hned po úspěšném vydání, si stěžují na neefektivitu, a že tráví příliš mnoho času řešením konfliktů při sloučení kódu. Implementace nové funkce vyžaduje, abyste změnili stejnou komplexní třídu, která je v rozporu s kódem vytvořeným jinými lidmi.



Řešení

Návrhový vzor **Strategy** navrhuje, abyste si vzali třídu, která dělá něco konkrétního mnoha různými způsoby, a extrahujte všechny tyto algoritmy do samostatných tříd nazývaných strategie.

Původní třída, nazývaná kontext, musí mít pole pro uložení odkazu na jednu ze strategií. Kontext deleguje práci na propojený strategický objekt namísto toho, aby ji vykonával samostatně.



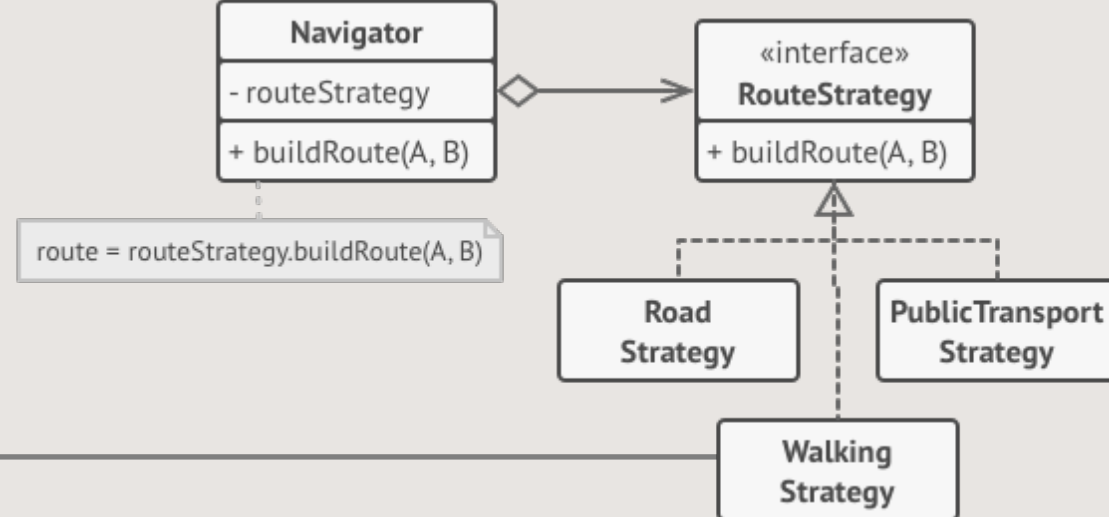
Řešení

Kontext není zodpovědný za výběr vhodného algoritmu pro danou úlohu. Místo toho klient předá požadovanou strategii kontextu. Ve skutečnosti kontext o strategiích mnoho neví. Pracuje se všemi strategiemi prostřednictvím stejného obecného rozhraní, které odhaluje pouze jedinou metodu pro spuštění algoritmu zapouzdřeného ve vybrané strategii.

Tímto způsobem se kontext stává nezávislým na konkrétních strategiích, takže můžete přidávat nové algoritmy nebo upravovat ty stávající, aniž byste měnili kód kontextu nebo jiné strategie.



Řešení



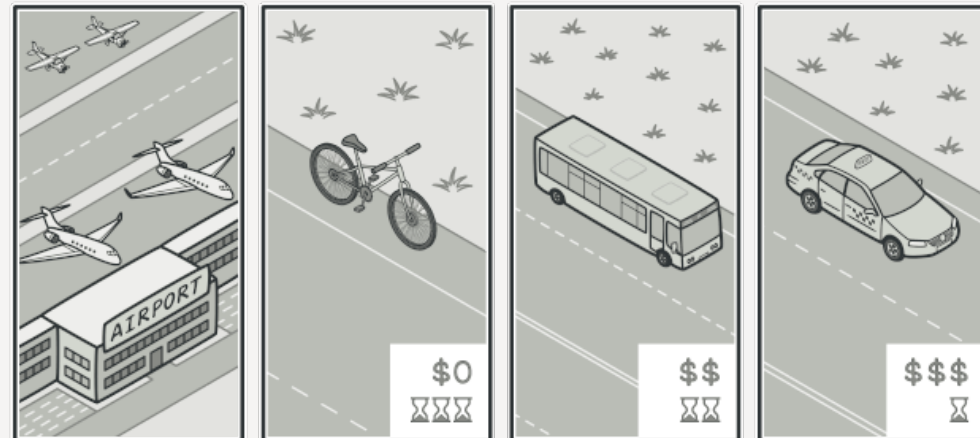
V naší navigační aplikaci lze každý směrovací algoritmus extrahovat do své vlastní třídy pomocí jediné metody **buildRoute**. Metoda přijímá počáteční a koncový bod trasy a vrací kolekci kontrolních bodů trasy.

I když se stejnými argumenty může každá třída směrování vytvořit jinou trasu, hlavní třídě navigátoru je ve skutečnosti jedno, který algoritmus je vybrán, protože její primární úlohou je vykreslit sadu kontrolních bodů na mapě. Třída má metodu pro přepínání aktivní strategie směrování, takže její klienti, jako jsou tlačítka v uživatelském rozhraní, mohou nahradit aktuálně zvolené chování směrování jiným.



Analogie v reálném světě

Představte si, že se musíte dostat na letiště. Můžete jet autobusem, objednat si taxi nebo nasednout na kolo. Toto jsou vaše dopravní strategie. Můžete si vybrat jednu ze strategií v závislosti na faktorech, jako je rozpočet nebo časová omezení.





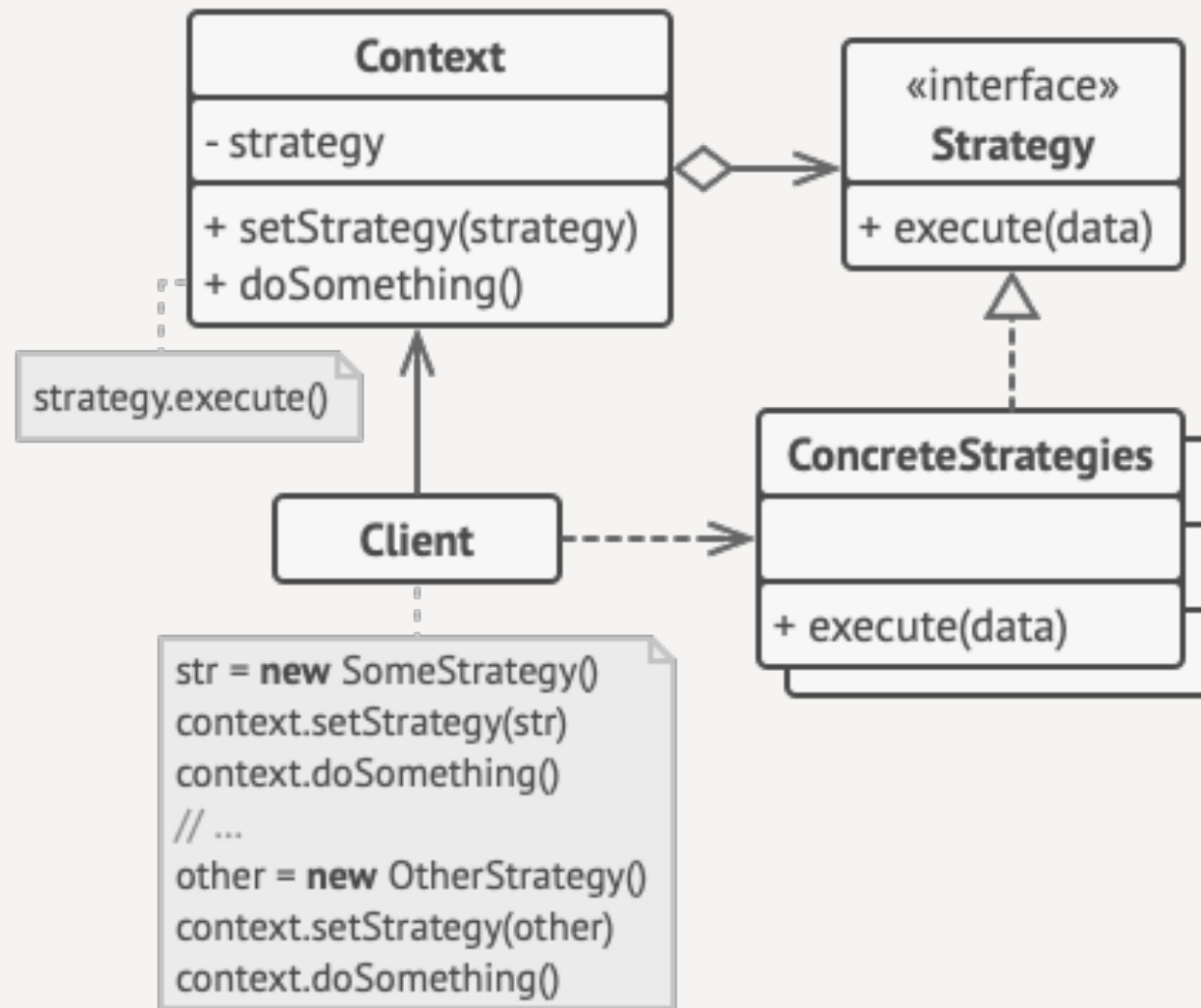
Výhody a nevýhody



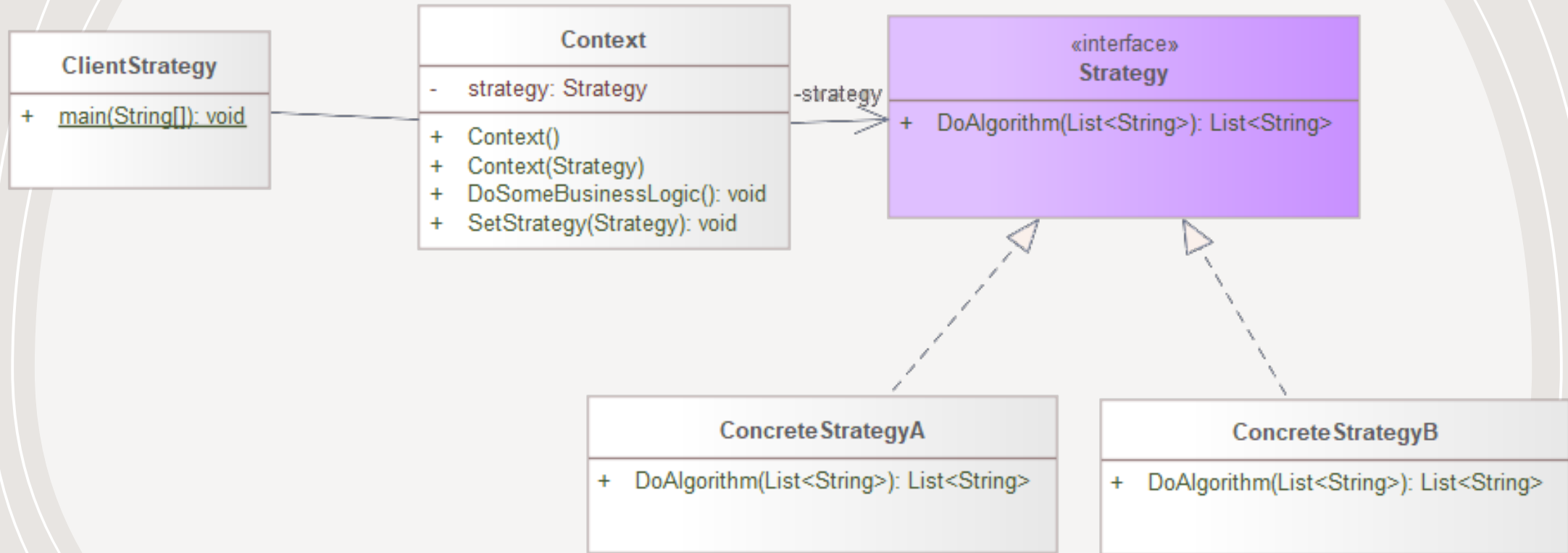
- Algoritmy používané uvnitř objektu můžete za běhu zaměnit.
- Podrobnosti implementace algoritmu můžete izolovat od kódu, který jej používá.
- Dědičnost můžete nahradit složením.
- Princip otevřený/uzavřený. Můžete zavádět nové strategie, aniž byste museli měnit kontext.



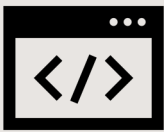
- Pokud máte jen pár algoritmů a ty se mění jen zřídka, není žádný skutečný důvod příliš komplikovat program novými třídami a rozhraními, které přicházejí spolu se vzorem.
- Klienti si musí být vědomi rozdílů mezi strategiemi, aby si mohli vybrat tu správnou.



Struktura



Implementace



```
public interface Strategy {  
    List<String> DoAlgorithm(List<String> data );  
}
```

```
public class ConcreteStrategyA implements Strategy {  
    @Override  
    public List<String> DoAlgorithm(List<String> data) {  
        Collections.sort(data);  
        return data;  
    }  
}
```

```
public class ConcreteStrategyB implements Strategy {  
    @Override  
    public List<String> DoAlgorithm(List<String> data) {  
        Collections.sort(data);  
        Collections.reverse(data);  
        return data;  
    }  
}
```



```
// The Context defines the interface of interest to clients.
class Context
{
    // The Context maintains a reference to one of the Strategy objects. The
    // Context does not know the concrete class of a strategy. It should
    // work with all strategies via the Strategy interface.
    private Strategy strategy;

    public Context()
    { }

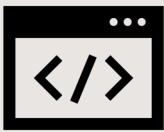
    // Usually, the Context accepts a strategy through the constructor, but
    // also provides a setter to change it at runtime.
    public Context(Strategy strategy)
    {
        this.strategy = strategy;
    }

    // Usually, the Context allows replacing a Strategy object at runtime.
    public void SetStrategy(Strategy strategy)
    {
        this.strategy = strategy;
    }

    // The Context delegates some work to the Strategy object instead of
    // implementing multiple versions of the algorithm on its own.
    public void DoSomeBusinessLogic()
    {
        System.out.println("Context: Sorting data using the strategy (not sure how it'll do it)");
        List<String> result = this.strategy.DoAlgorithm(new ArrayList<>(Arrays.asList("c", "b", "e", "a", "d" )))

        String resultStr = "";
        for (String element : result)
        {
            resultStr += element + ",";
        }

        System.out.println(resultStr);
    }
}
```



```
public class ClientStrategy {  
    public static void main(String[] args) {  
        Context context=new Context();  
  
        System.out.println("Client: Strategy is set to normal sorting.");  
        context.SetStrategy(new ConcreteStrategyA());  
        context.DoSomeBusinessLogic();  
  
        System.out.println();  
  
        System.out.println("Client: Strategy is set to reverse sorting.");  
        context.SetStrategy(new ConcreteStrategyB());  
        context.DoSomeBusinessLogic();  
    }  
}
```



Zdroje

<https://refactoring.guru/design-patterns>

<http://voho.eu/wiki/navrhovy-vzor/>

<https://www.oodesign.com/>